Modelado de Datos con Estructuras y Punteros: Struct, Operador Flecha y Pasaje por Referencia para el Proyecto Integrador Anual

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B

Prof. Mansilla Muñoz York Elías

2026

Resumen

Se modela una entidad del mundo real —sensor o módulo— con la estructura *EntidadProyecto* (id, nombre, métrica) y se opera por referencia mediante puntero y operador flecha (->). El pasaje por dirección evita copias costosas y permite que la función *cargarDatos* modifique el objeto original en memoria RAM. Se documenta el rol crítico de *cin.ignore()* para limpiar el búfer antes de *getline*. Las pruebas con identificador 101 y métrica decimal validan la persistencia en memoria y la impresión de la dirección hexadecimal. El patrón es directamente reutilizable para las variables del Proyecto Integrador Anual de la Expotécnica.

Palabras clave: struct, operador flecha, puntero, pasaje por referencia, búfer, cin.ignore, modelado de datos

Abstract

A real-world entity—sensor or module—is modeled with the *EntidadProyecto* structure (id, name, metric) and operated by reference via pointer and arrow operator (->). Passing by address avoids costly copies and allows *cargarDatos* to modify the original object in RAM. The critical role of *cin.ignore()* to flush the buffer before *getline* is documented. Tests with ID 101 and a decimal metric validate memory persistence and hexadecimal address printing. The pattern is directly reusable for Annual Integrative Project variables for the technical exhibition.

Keywords: struct, arrow operator, pointer, pass by reference, buffer, cin.ignore, data modeling

Introducción

El Proyecto Integrador Anual exige representar hardware y procesos con datos heterogéneos. Una estructura (*struct*) agrupa esos campos en una sola entidad lógica contigua en memoria RAM; un puntero permite a las funciones trabajar sobre el original sin

duplicar memoria.

La Actividad 5 proporciona el molde: *EntidadProyecto* con identificador, nombre y métrica, y la función *cargarDatos(EntidadProyecto* ptr)* que emplea el operador flecha. El objetivo es dominar definición, pasaje por dirección y gestión del búfer.

Revisión de la Literatura

La estructura es una colección heterogénea en bloque contiguo, a diferencia de variables sueltas dispersas. El operador flecha es azúcar sintáctica para *(*ptr).campo* y solo es válido con punteros. El pasaje por referencia mediante dirección consiste en enviar *&objeto* para modificar el original; el pasaje por copia duplicaría la estructura. El búfer de entrada es la cola donde quedan caracteres; el salto de línea de la tecla Enter puede ser leído por *getline* como línea vacía, y *cin.ignore()* descarta ese carácter pendiente (GeeksforGeeks, 2024; cplusplus.com, 2024).

Notación	Cuándo usar	Ejemplo
Punto (.)	Objeto directo	miEntidad.id
Flecha (->)	Puntero a struct	ptr->id ≡ (*ptr).id

Nota. Acceso a miembros según tipo de variable.

Método

Se definió *struct EntidadProyecto { int id; char nombre[50]; float metrica; };* con inicialización segura *{0, "Vacio - Aramis Demaria", 0.0f}* en la pila. La invocación se realizó como *cargarDatos(&miEntidad)*, entregando la dirección hexadecimal. En el interior se ejecutó *cin >> ptr->id; cin.ignore(); cin.getline(ptr->nombre, 50); cin >> ptr->metrica;*.

La verificación imprimió los campos y la dirección *&miEntidad* en hexadecimal. La estructura */sistemaproyecto* contiene *docs/manuales/* para el manual del programador, *capturas/ejecucion_struct.png* y *src/main.cpp*. El versionado siguió el esquema SemVer y DocVer *v1.0.0 Baseline*, con ramas *main* y *development* para el Proyecto Integrador Anual.

Resultados

La entrada 101, “Sensor DHT22 - Invernadero” y 23.7 se reflejó idénticamente en la verificación en memoria RAM y la dirección hexadecimal se mostró correctamente (por ejemplo, *0x61fe18*). Omitir *cin.ignore()* reprodujo el defecto documentado: *getline* leyó una línea vacía.

Figura 1. Ejecución completa con operador flecha (capturas/ejecucion_struct.png).

Discusión

Agrupar en estructura aporta cohesión semántica y localidad de caché; las variables sueltas son propensas a desincronización y el pasaje de cinco parámetros sueltos es frágil. La notación de punto con puntero (*ptr.id*) constituye error de tipos: el puntero no es una estructura sino una dirección; el compilador exige el operador flecha para desreferenciar y acceder.

El pasaje por copia copiaría más de cincuenta bytes por llamada; por referencia solo se copian ocho bytes de la dirección y la función opera sobre el original, con ahorro de memoria y sin copias desactualizadas. Sin *ignore()*, el salto de línea residual hace que *getline* consuma una línea vacía y omita la pregunta de nombre, defecto clásico al combinar el operador >> con *getline*.

Conclusión

El modelado con estructura, puntero y operador flecha quedó operativo y documentado en los manuales. Constituye el patrón recomendado para las variables del Proyecto Integrador Anual (sensores, stock, usuarios). Como paso siguiente se propone persistir la estructura en disco con flujos de archivo (Actividad 7) y evolucionar hacia clases con encapsulamiento (Actividad 8).

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

cplusplus.com. (2024). *Data structures (struct).*

https://cplusplus.com/doc/tutorial/structures/

GeeksforGeeks. (2024). *Pointers and structures in C++*.

https://www.geeksforgeeks.org/pointers-structures-cpp/

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.